

AWS Fargate



CCBDA Q1 - MEI UPC
Pol Plana - Zixin Zhang -
Zhehan Xiang - Zhipeng Lin

1
0
0
1
0
1
0
1
0
1
0
1
0
1
0
0

1
0
1
0
0
1
0
1
0
1
0
0
1
0
1
0



Table of contents

01

**Introducción y
Relevancia**

03

**“Hands-on”
Teórico: Tutorial**

02

**Arquitectura y
Comparativa**

04

**Perspectiva de
Costes y Recursos**

1
0
1
0
1
0
1
1
1
0
0
0
0
1
1
1
0

1
1
0
0
1
0
1
0
1
0
1
0
1
1
1
0

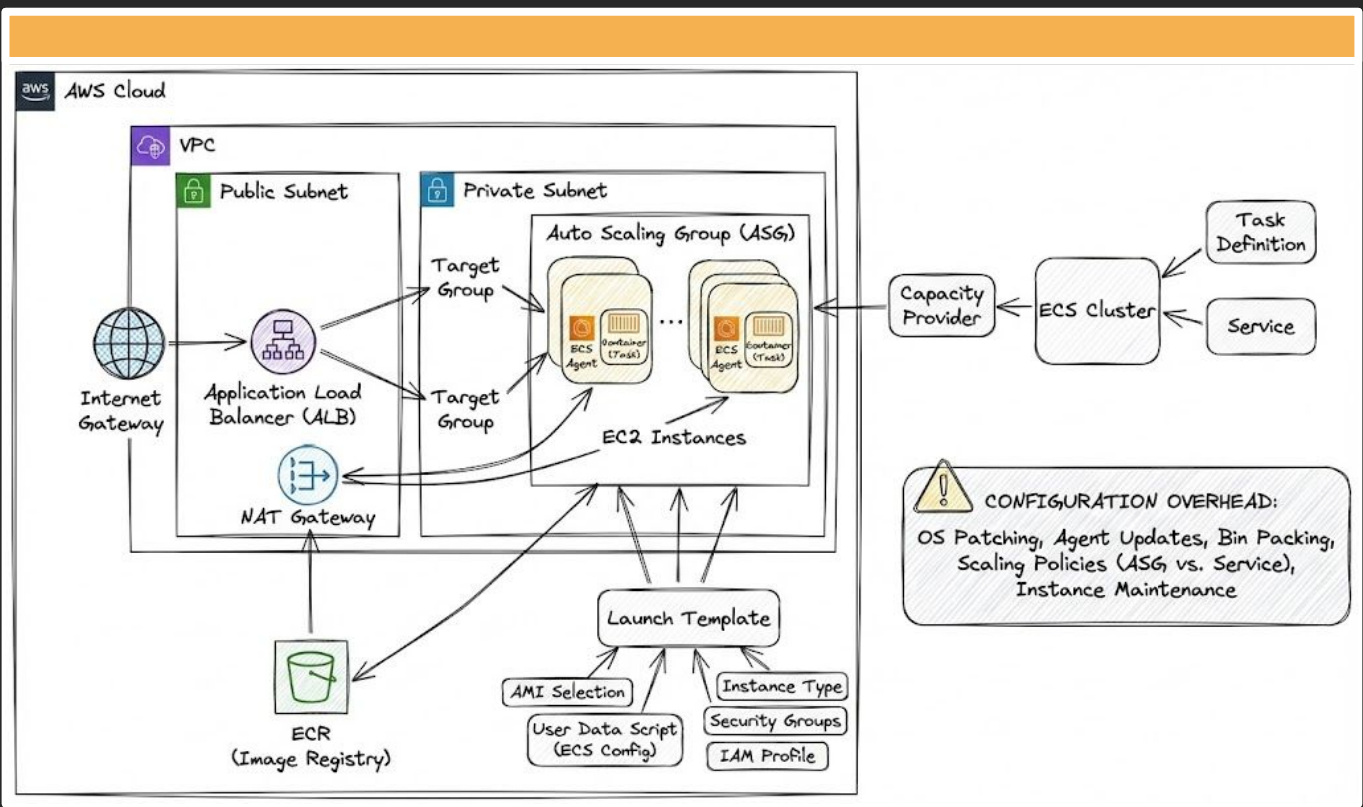


Introducción y Relevancia

1
0
1
0
1
0
1
0
1
1
1
0
0
0
0
1
1
1
1
0

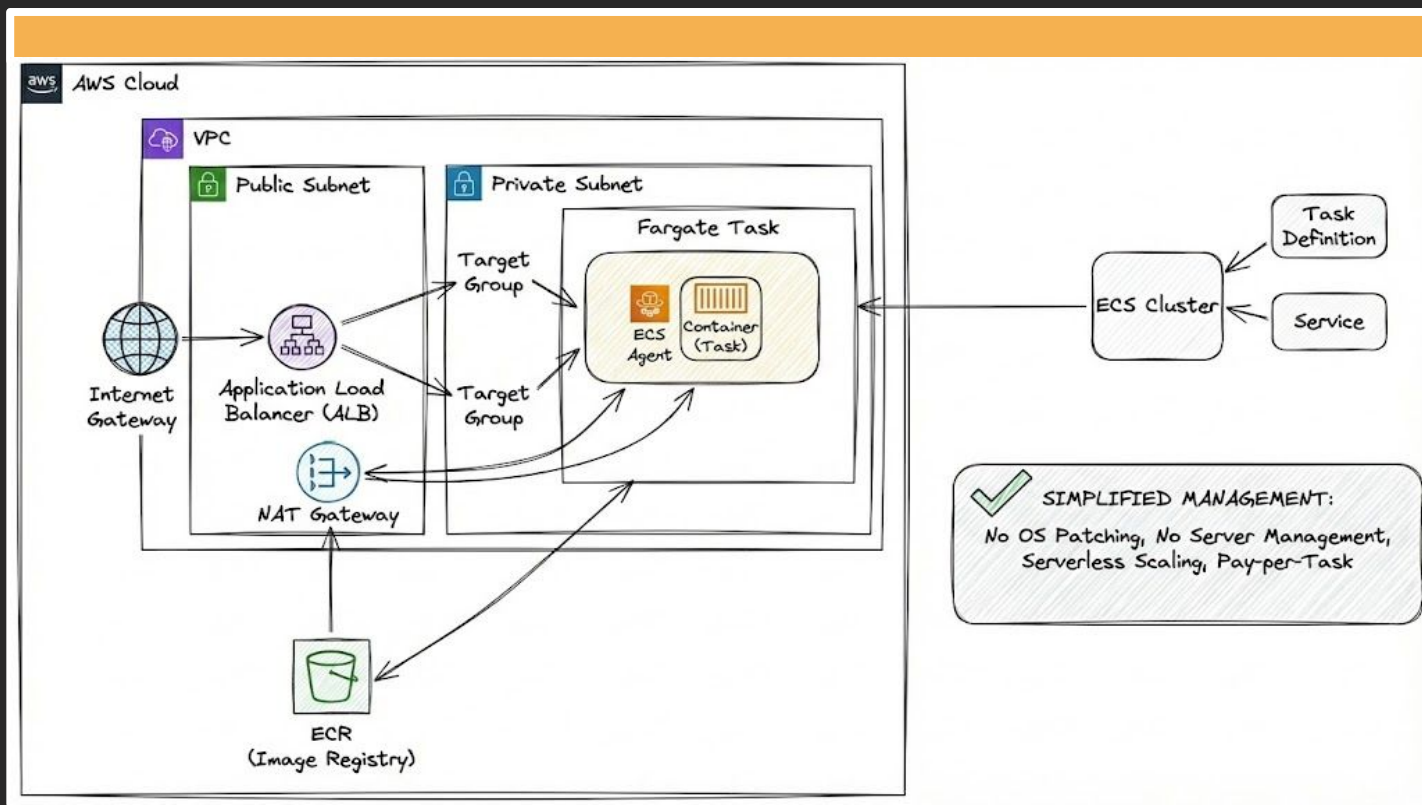
SERVERLESS COMPUTE FOR CONTAINER

1
1
0
0
1
0
1
0
1
0
1
0
1
0
1
1
1
0



1
0
1
0
1
0
1
1
1
0
0
0
1
1
1
0

1
1
0
0
1
0
1
0
1
0
1
0
1
1
1
0



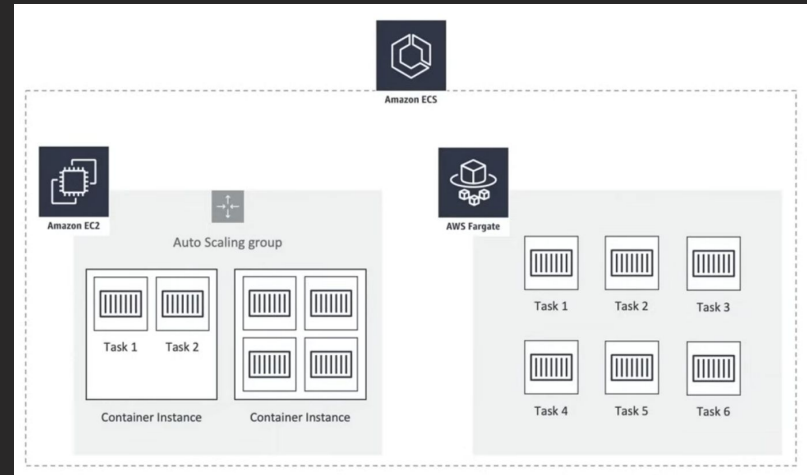
1
0
1
0
1
0
1
1
1
0
0
0
1
1
1
0

1
1
0
0
1
0
1
0
1
0
1
1
1
0

EC2 vs. Fargate

Modelo EC2: Eres dueño de la VM, pagas por capacidad reservada (aunque no la uses), tienes control total pero responsabilidad total.

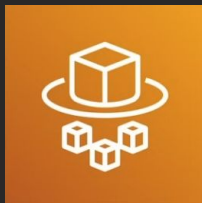
Modelo Fargate: AWS gestiona toda la infraestructura. Pagas por vCPU/RAM consumida, mayor aislamiento de seguridad.



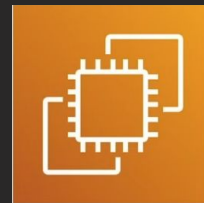
Arquitectura y Comparativa

¿Cuándo usar cuál?

Fargate: Cargas de trabajo variables, entornos de desarrollo, o cuando no quieres administrar SO.



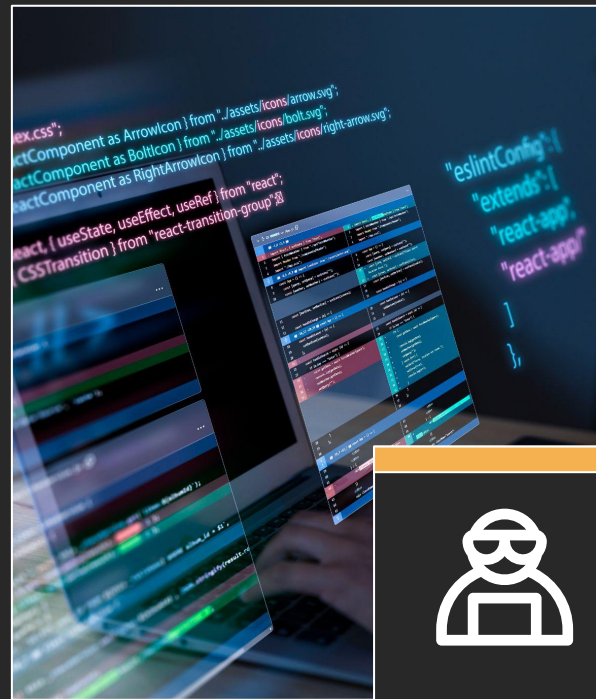
EC2: Cargas muy constantes (coste), control profundo del hardware (GPUs específicas), o Reserved Instances.

1
0
0
1
0
1
0
1
0
0
1
0
1
0
1
01
0
1
0
0
1
0
0
1
0
1
1
0
1
1
1

“Hands-on” Teórico: Tutorial

ECS Key Terms

- **Task** - The lowest level building block of ECS - Runtimes instances
- **Task Definitions** - Templates for your **Tasks**. This is where you specify your Docker image, Memory/CPU Requirements, etc.
- **Container (EC2 Only)** - Virtualized Instance that run your **Tasks**.
- **Cluster** - EC2 - A group of Containers which run **Tasks**
Fargate - A group of **Tasks**
- **Service** - A Task management system that ensures X amount of tasks are up and running.



“Hands-on” Teórico: Tutorial

Preparamos nuestro proyecto y lo Dockerizamos

app.py code

Dockerfile code

```
code > app.py > ...
1 from flask import render_template
2 from flask import Flask
3
4 app = Flask(__name__)
5
6
7
8
9 @app.route('/')
10 def home():
11     return render_template('index.html')
12
13 @app.route('/app')
14 def blog():
15     return "Hello, from App!"
16
17
18
19 if __name__ == '__main__':
20     app.run(threaded=True, host='0.0.0.0', port=8081)
21
22
```

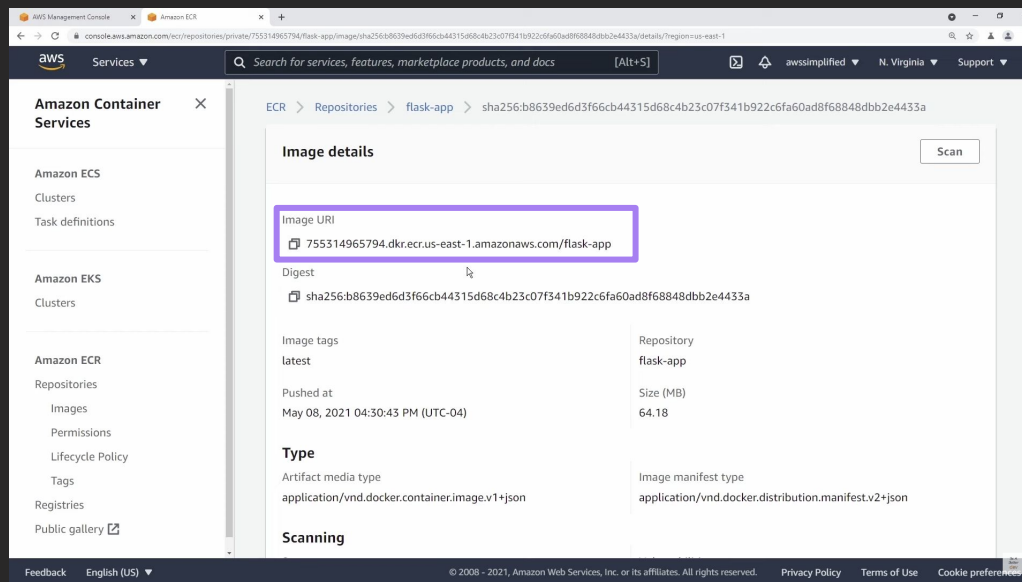
```
code > Dockerfile > FROM
1 FROM python:3.7-slim
2
3 COPY ./requirements.txt /app/requirements.txt
4
5 WORKDIR /app
6
7 RUN pip install -r requirements.txt
8
9 COPY . /app
10
11 EXPOSE 8081
12
13 ENTRYPOINT [ "python" ]
14
15 CMD [ "app.py" ]
16
17
```

1
0
1
0
0
1
0
1
0
1
0
1
0
0

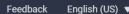
1
0
0
1
0
1
0
1
0
1
0
1
0
0

“Hands-on” Teórico: Tutorial

Tenemos nuestra imagen de Docker almacenada en ECR



1
0
0
1
0
1
0
1
0
1
0
1
0
1
0
0

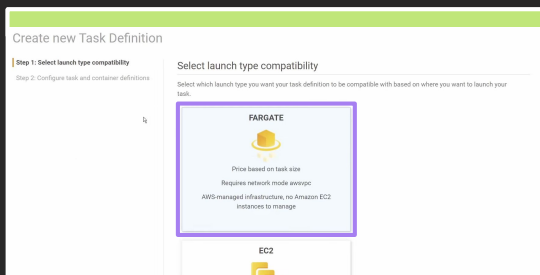


1
0
1
0
0
1
0
1
0
0
0
1
0
1
0

Create VPC ☐ Create a new VPC for this cluster

- Amazon ECS Clusters
- Task Definitions**
- Account Settings
- Amazon EKS Clusters
- Amazon ECR Repositories
- AWS Marketplace Discover software
- Subscriptions

Ahora definimos una tarea con nuestro proyecto Dockerizado y almacenado en ECR



Task Definition Name*

Task memory (GB)

2GB

The valid memory range for 1 vCPU is: 2GB - 8GB.

Task CPU (vCPU)

1 vCPU

The valid CPU range for 2GB memory is: 0.25 vCPU - 1 vCPU.

Task memory maximum allocation for container memory reservation

0

2048 shared of 2048 MiB

Task CPU maximum allocation for containers

0

1024 shared of 1024 CPU units

The screenshot shows the Docker Desktop interface. At the top, there is a blue button labeled "Add container". Below it, a mouse cursor is pointing at the button. Underneath the button is a table with three columns: "Container ...", "Image", and "Hard/S...". The table is currently empty.

Create Task Definition: DemoTaskDefinition

DemoTaskDefinition succeeded

Create CloudWatch Log Group

✔ CloudWatch Log Group created
CloudWatch Log Group /ecs/DemoTaskDefinition

Add container

▼ Standard

Container name* DemoAppContainer

Image* 755314965794.dkr.ecr.us-east-1.amazonaws.com/flask-app:latest

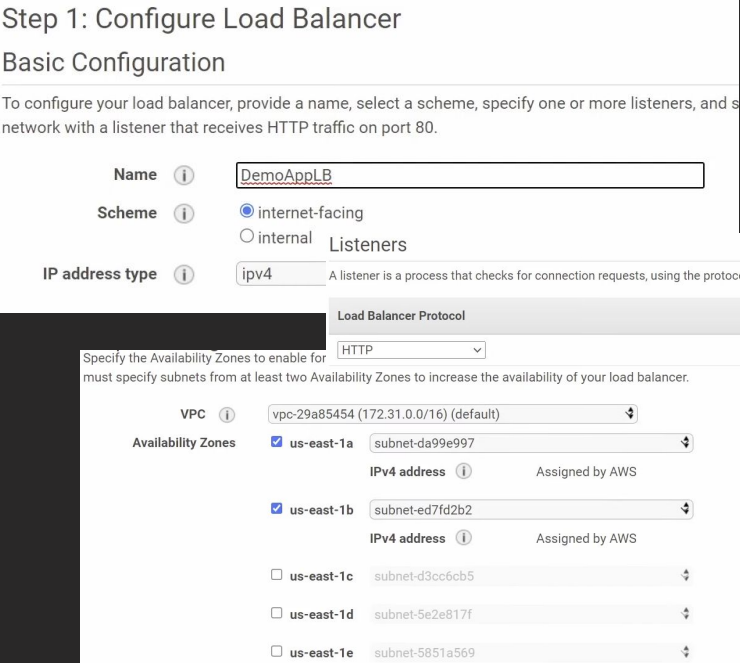
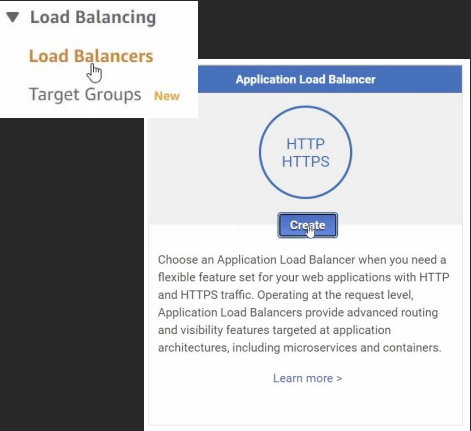
Memory Limits (MiB) Soft limit ▼ 2048

Port mappings	Container port	Protocol
	8081	tcp

1
0
1
0
0
1
0
1
0
1
0
0
1
0
0
1
0

“Hands-on” Teórico: Tutorial

A continuación creamos un balanceador de carga a través del servicio EC2



“Hands-on” Teórico: Tutorial

No necesitamos hacer nada en la step 2, ya que hemos usado los valores por defecto (HTTP). No se ha configurado HTTPS en este entorno de prueba.

1. Configure Load Balancer

2. Configure Security Settings

3. Configure Security Groups

4. Configure Routing

5. Register Targets

6. Review

Step 3: Configure Security Groups

A security group is a set of firewall rules that control the traffic to your load balancer. On this page, you can add rules to allow specific traffic to reach your load balancer. First, decide whether to create a new security group or select an existing one.

Assign a security group

☒ Create a new security group

☐ Select an existing security group

Security group name

DemoAppLB-SecurityGroup

Description

DemoAppLB-SecurityGroup

Type i	Protocol i	Port Range i	Source i	
Custom TCP R ▾	TCP	80	Custom ▾ 0.0.0.0/0, ::/0	✕

1
0
0
1
0
1
0
1
0
1
0
1
0
1
0
0

1
0
1
0
0
1
0
1
0
1
0
0
1
0
1
0

“Hands-on” Teórico: Tutorial

No necesitamos hacer nada en la step 2, ya que hemos usado los valores por defecto (HTTP). No se ha configurado HTTPS en este entorno de prueba.

Step 4: Configure Routing

Your load balancer routes requests to the targets in this target group using the protocol and port you specify in this step will apply to all of the listeners configured on this load balancer.

Target group

Target group ⓘ

New target group ⌵

Name ⓘ

DemoAppTargetGroup

Target type

☐ Instance

☒ IP

☐ Lambda function

Protocol ⓘ

HTTP ⌵

Port ⓘ

8081

Tampoco vamos a registrar targets (step 5), ya que aún no tenemos nuestras instancias ejecutándose en Fargate

Create Load Balancer

Actions ▾

🔍 Filter by tags and attributes or search by keyword

☐	Name	DNS name	State
☐	DemoAppLB	DemoAppLB-1865590900.u...	provisioning

☐

DemoAppLB

DemoAppLB-1865590900.u...

active

1
0
0
1
0
1
0
1
0
1
0
1
0
0

1
0
1
0
0
1
0
1
0
1
0
0
1
0
1
0

“Hands-on” Teórico: Tutorial

The screenshot shows the AWS Management Console interface for an Amazon ECS cluster named 'DemoAppCluster'. The left sidebar lists navigation options: Amazon ECS, Clusters, Task Definitions, Account Settings, Amazon EKS, Clusters, Amazon ECR, Repositories, AWS Marketplace, Discover software, and Subscriptions. The main content area shows the cluster details, including the Cluster ARN, Status (ACTIVE), and Registered container instances (0). Below this, there are tabs for Services, Tasks, ECS Instances, Metrics, Scheduled Tasks, Tags, and Capacity Providers. The 'Services' tab is selected, showing a 'Create' button and a table with columns for Service Name, Status, Service, Task De..., Desired..., Runnin..., Launch ..., and Platfor....

Cluster : DemoAppCluster

Get a detailed view of the resources on your cluster.

Cluster ARN: arn:aws:ecs:us-east-1:755314965794:cluster/DemoAppCluster

Status: ACTIVE

Registered container instances: 0

Pending tasks count: 0 Fargate, 0 EC2

Running tasks count: 0 Fargate, 0 EC2

Active service count: 0 Fargate, 0 EC2

Draining service count: 0 Fargate, 0 EC2

Services | Tasks | ECS Instances | Metrics | Scheduled Tasks | Tags | Capacity Providers

Create | Update | Delete | Actions

Last updated on May 8, 2021 9:31:36 PM (0m ago)

Filter in this page | Launch type: ALL | Service type: ALL

	Service Name	Status ...	Service ...	Task De...	Desired...	Runnin...	Launch ...	Platfor...
No results								

Ya podemos crear
nuestro servicio en el
clúster creado
previamente en ECS

“Hands-on” Teórico: Tutorial

Step 1: Configure service

Step 2: Configure network

Step 3: Set Auto Scaling (optional)

Step 4: Review

Launch type ☒ FARGATE ☐ EC2 ⓘ

[Switch to capacity provider strategy](#) ⓘ

Task Definition Family
DemoTaskDefinition ▼

Revision
1 (latest) ▼

Platform version
LATEST ⓘ

Cluster
DemoAppCluster ⓘ

Service name
DemoAppService ⓘ

Service type*
REPLICA ⓘ

Number of tasks
2 ⓘ

Number of tasks represents the minimum tasks that will be running. A maximum can be set through autoscaling.

“Hands-on” Teórico: Tutorial

Step 2: Configure network

Configure network

VPC and security groups

VPC and security groups are configurable when your task definition uses the awsvpc network mode.

Cluster VPC* ⓘ

Subnets* ⓘ

subnet-da99e997
(172.31.16.0/20) - us-east-1a
assign ipv6 on creation: Disabled

subnet-ed7fd2b2
(172.31.32.0/20) - us-east-1b
assign ipv6 on creation: Disabled

Security groups* DemoAp-6734 ⓘ

Auto-assign public IP ⓘ

Load balancer type*

☐ None
Your service will not use a load balancer.

☒ **Application Load Balancer**
Allows containers to use dynamic host port mapping (multiple tasks allowed per container instance). Multiple services can use the same listener port on a single load balancer with rule-based routing and paths.

☐ Network Load Balancer
A Network Load Balancer functions at the fourth layer of the Open Systems Interconnection (OSI) model. After the load balancer receives a request, it selects a target from the target group for the default rule using a flow hash routing algorithm.

☐ Classic Load Balancer
Requires static host port mappings (only one task allowed per container instance); rule-based routing and paths are not supported.

Service IAM role Task definitions that use the awsvpc network mode use the AWSServiceRoleForECS service-linked role, which is created for you automatically. [Learn more.](#)

Load balancer name

Container to load balancer

Container name : port

1
0
0
1
0
1
0
1
0
1
0
1
0
1
0
1
0
0

1
0
1
0
0
1
0
1
0
0
1
0
1
0
1
0

“Hands-on” Teórico: Tutorial

Container to load balance

Container name : port DemoAppContainer:8081:8081

Add to load balancer

Container to load balance

DemoAppContainer : 8081 Remove ✕

Production listener port* 80:HTTP ⓘ

Production listener protocol* HTTP

Target group name DemoAppTargetGroup ⓘ

Target group protocol HTTP ⓘ

Target type ip ⓘ

Path pattern / Evaluation order default

Health check path / ⓘ

Additional health check options can be configured in the ELB console after you create your service.

Si quisiéramos que el número de tareas definido anteriormente se ajuste a la demanda, podríamos configurarlo.

Step 3: Set Auto Scaling (optional)

Set Auto Scaling (optional)

Automatically adjust your service's desired count up and down within a specified range in response to changes in demand. You can modify your Service Auto Scaling configuration at any time to meet the needs of your application.

Service Auto Scaling ☒ Do not adjust the service's desired count

☐ Configure Service Auto Scaling to adjust your service's desired count

A continuación creamos el servicio.

1
0
0
1
0
1
0
1
0
1
0
1
0
1
0
0

1
0
1
0
0
1
0
1
0
1
0
0
1
0
1
0

1
0
0
1
0
1
0
1
0
1
0
1
0
1
0
0

1
0
1
0
0
1
0
1
0
1
0
0
1
0
1
0

Task	Task Definition ...	Last status
65584627712247...	DemoTaskDefiniti...	PENDING
d6b0d739b4b842...	DemoTaskDefiniti...	PROVISIONING

Hemos creado el servicio con sus dos tareas. Estas pasarán de pendiente a listo, y después fallarán. Necesitamos permitir que se conecten a nuestra red, con el Load Balancer, ya que este les hará "health pings", pero no responderán, se pensará que no existen y se recrearán de nuevo infinitamente.

10100101001010

10010101010100

Security Groups (1/1)
Info

Refresh
Actions
Create security group

☒	Name	Security group ID	Security group name	VPC ID	Description
sg-0827ce7c831e00105 - DemoAp-6734					
Details Inbound rules Outbound rules Tags					

Inbound rules (1)

Type	Protocol	Port range	Source	Description - optional
HTTP	TCP	80	0.0.0.0/0	-

Edit inbound rules

“Hands-on” Teórico: Tutorial

Accedemos a los detalles de nuestro Cluster de ECS y seleccionamos el Security group configurado para ver sus detalles. Vamos a editar las inbound rules.

DetailsTasksEventsAuto ScalingDeploymentsMetricsTags

Load Balancing

Target Group Name	Container Name	Container Port
DemoAppTargetGroup	DemoAppContainer	8081

Network Access

Health check grace period

0

Allowed VPC

vpc-29a85454

Allowed subnets

subnet-da99e997,subnet-ed7fd2b2

Security groups*

sg-0827ce7c831e00105

Auto-assign public IP

ENABLED

Security Groups (1/1) Info

Filter security groups

search: sg-0827ce7c831e00105 X Clear filters

Name	Security group ID	Security group name	VPC ID	Description
sg-0827ce7c831e00105 - DemoAp-6734				
DetailsInbound rulesOutbound rulesTags				
Inbound rules (1)				
Type	Protocol	Port range	Source	Description - optional
HTTP	TCP	80	0.0.0.0/0	-

Edit inbound rules

Type InfoProtocol InfoPort range InfoSource InfoDescription - optional Info

All TCP TCP 0 - 65535 Custom Q sg-0f7fd1b9338114e8f Del etc

Inbound security group rules successfully modified on security group (sg-0827ce7c831e00105 | DemoAp-6734)

1
0
1
0
0
1
0
1
0
1
0
0
1
0
1
0

“Hands-on” Teórico: Tutorial

The screenshot displays the AWS Management Console interface for configuring a Load Balancer. On the left, a navigation menu lists services: AMIs, Elastic Block Store (with sub-items Volumes, Snapshots, and Lifecycle Manager), Network & Security (with sub-items Security Groups, Elastic IPs, and Placement Groups), and Load Balancing (with sub-items Load Balancers and Target Groups). The 'Load Balancing' section is selected.

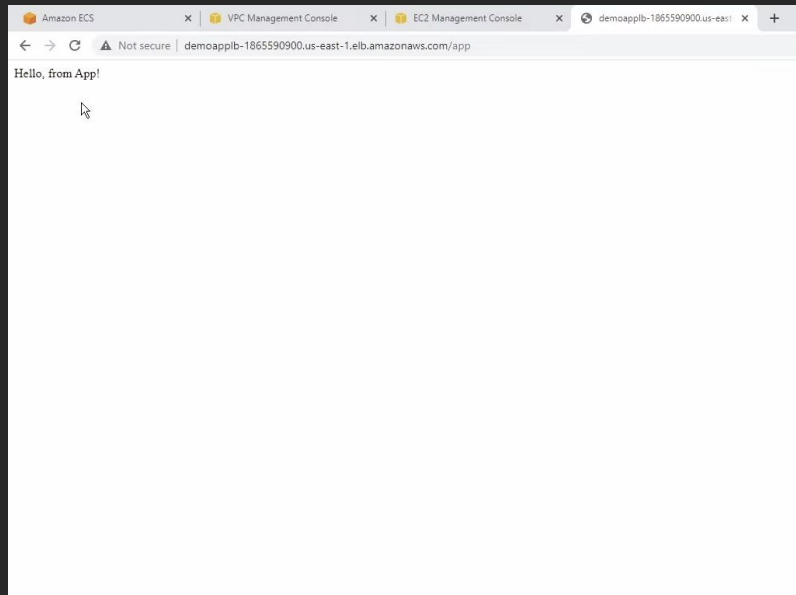
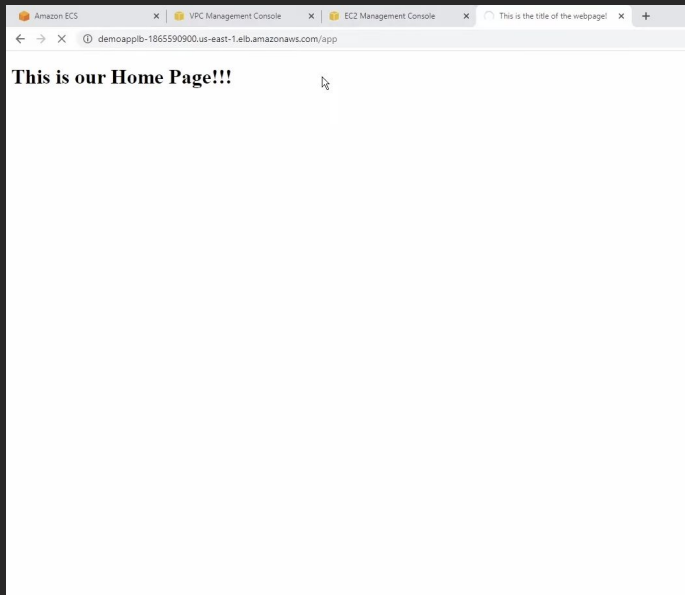
The main content area shows the 'Create Load Balancer' button and an 'Actions' dropdown. Below this is a search bar and a table of existing load balancers. The table has columns for Name, DNS name, and State. One load balancer, 'DemoAppLB', is listed with the DNS name 'DemoAppLB-1865590900.u...' and is in an 'active' state.

Below the table, the 'Load balancer: DemoAppLB' section is visible. It contains tabs for Description, Listeners, Monitoring, Integrated services, and Tags. The 'Description' tab is active, showing the 'Basic Configuration' section. This section lists the following details:

- Name: DemoAppLB
- ARN: arn:aws:elasticloadbalancing:us-east-1:755314965794:loadbalanc...
- DNS name: DemoAppLB-1865590900.us-east-1.elb.amazonaws.com (A Record)
- State: active
- Type: application

The 'DNS name' field is highlighted with a red rectangle, indicating it is the focus of the tutorial.

“Hands-on” Teórico: Tutorial



1
0
0
1
0
1
0
1
0
1
0
1
0
1
0
0

1
0
1
0
0
1
0
1
0
1
0
0
1
0
1
0

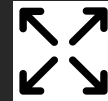
Modelos de precios



**Pagas por CPU y
GB de memoria
por segundo.**



**Cuidado con
dejar tareas
zombies
encendidas.**

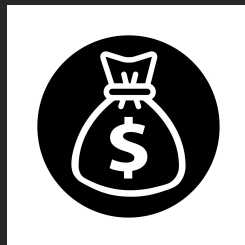
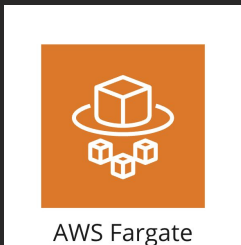


Right-Sizing

1
0
0
1
0
1
0
1
0
0
1
0
1
0
1
0

1
0
1
0
0
1
0
0
1
0
1
1
0
1
1
1

Comparativa de TCO: Fargate vs EC2



Costes Operativos Ocultos

EC2:

- Parchear OS
- Agentes de Docker
- Escalar cluster
- (...)

Fargate: Null

Eficiencia de uso

EC2: Espacio para picos.

Fargate: 100%

1
0
0
1
0
1
0
1
0
0
1
0
1
0
1
0
1
0

1
0
1
0
0
1
0
0
1
0
1
0
1
1
1
1

Recursos de aprendizaje

Página oficial: <https://aws.amazon.com/es/fargate/>

Workshop: <https://awsworkshop.io/tags/fargate/>

Otros: <https://spacelift.io/blog/what-is-aws-fargate#what-is-aws-fargate>

<https://www.youtube.com/watch?v=DVrGXjjkpig>

<https://www.youtube.com/watch?v=o7s-eigrMAI>

1
0
0
1
0
1
0
1
0
0
0
1
0
1
0
1
0

1
0
1
0
0
1
0
0
1
0
1
1
0
1
1
1

Conclusión final

1
0
0
1
0
1
0
1
0
0
1
0
1
0
1
0
0

line

1
0
1
0
0
1
0
0
1
0
1
1
0
1
1
1
1



Gracias!

Pol Plana - Zixin Zhang -
Zhehan Xiang - Zhipeng Lin

**CCBDA Q1
MEI UPC**

1
0
0
1
0
1
0
1
0
0
1
0
1
0
1
0

1
0
1
0
0
1
0
0
1
0
1
1
0
1
1
1